TICKETRUSH · SERVER/

BACKEND ARCHITECTURE

How a request moves through the API, how sign-up, sign-in, refresh and logout work, and which security controls and rate limits protect each route.

EXPRESS 5 | MONGODB ATLAS · MONGOOSE 9 | TYPESCRIPT | NODE 24 | ZOD · JOSE · ARGON2

Last updated 7 Nov 2026 · verification emails at sign-up, and `POST /verify-email`

CONTENTS

01 System overview
02 Code layout and layers
03 Global middleware
04 Auth router
05 Routes and responses
06 Rate limits
07 Sign-up flow
08 Sign-in flow
09 Refresh and logout
10 Tokens and protected routes
11 Error handling
12 Data model
13 Startup and shutdown
14 Security controls
15 Configuration and testing
16 Known gaps and next steps

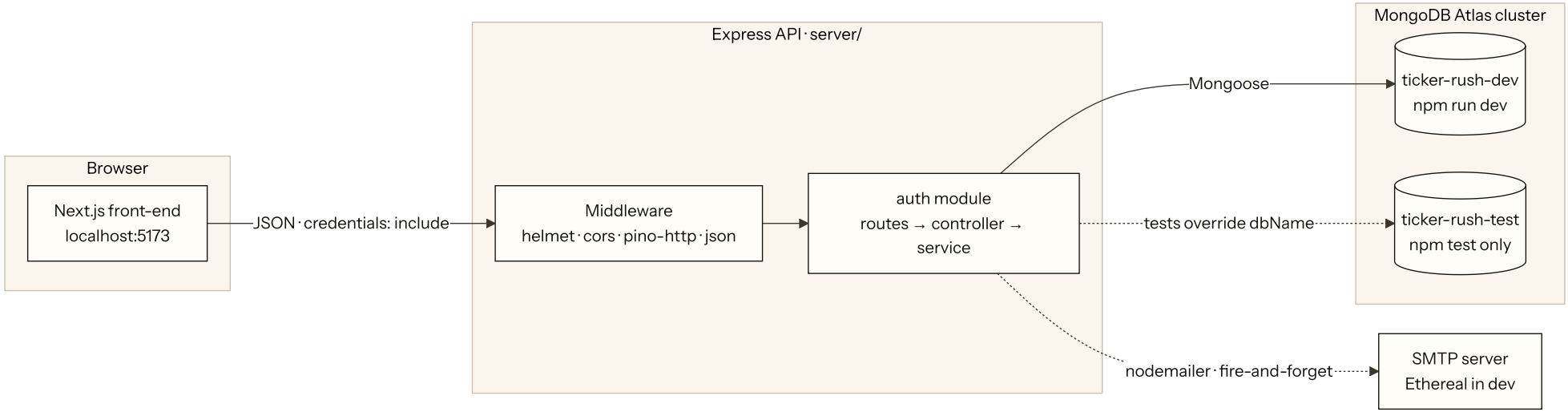
ADMIT ONE



N° 000001

01 SYSTEM OVERVIEW

Who talks to whom, and where the data lives.



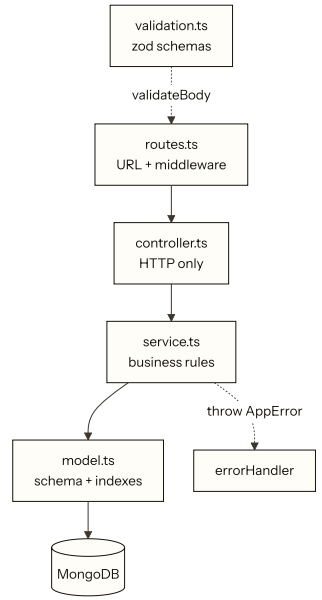
KEY POINTS

- The browser calls the API **directly**, cross-origin. The Next.js server is not in the request path.
- CORS lets exactly one origin in (`CORS_ORIGIN`), with credentials, so the refresh cookie can travel.
- Only the API touches MongoDB. Dev data is in `ticker-rush-dev`; `npm test` uses `ticker-rush-test`.
- Dev ports: front-end 5173, API 8080 (`PORT`, default 3000).

02 CODE LAYOUT AND LAYERS

Where things live, and the one-way flow between layers.

```
server/src/
├── server.ts                entry: connect DB → listen → shutdown
├── app.ts                   middleware, /health, /api/v1, errors
├── routes.ts                apiRouter: mounts each module
├── config/
│   ├── env.ts               zod-validated env → config
│   ├── database.ts          connect / disconnect, connection logs
│   ├── logger.ts            pino (redaction) + pino-http
│   └── mailer.ts            nodemailer transport, sendEmail
├── common/
│   ├── errors/app-error.ts  AppError(status, message, CODE)
│   ├── errors/error.middleware.ts  404 + errorHandler
│   └── middleware/validate.ts      validateBody(schema)
└── modules/auth/
    ├── auth.routes.ts       URLs, middleware chain, rate limits
    ├── auth.validation.ts   zod schemas
    ├── auth.types.ts        types inferred from schemas
    ├── auth.controller.ts   HTTP: status, headers, cookie
    ├── auth.service.ts      hashing, credentials, sessions
    ├── auth.tokens.ts       JWT sign/verify, sha256
    ├── auth.middleware.ts   requireAuth, requireRole
    ├── auth.model.ts        AuthAccount collection
    ├── auth-token.model.ts  AuthToken collection
    └── auth.test.ts         node:test integration tests
```

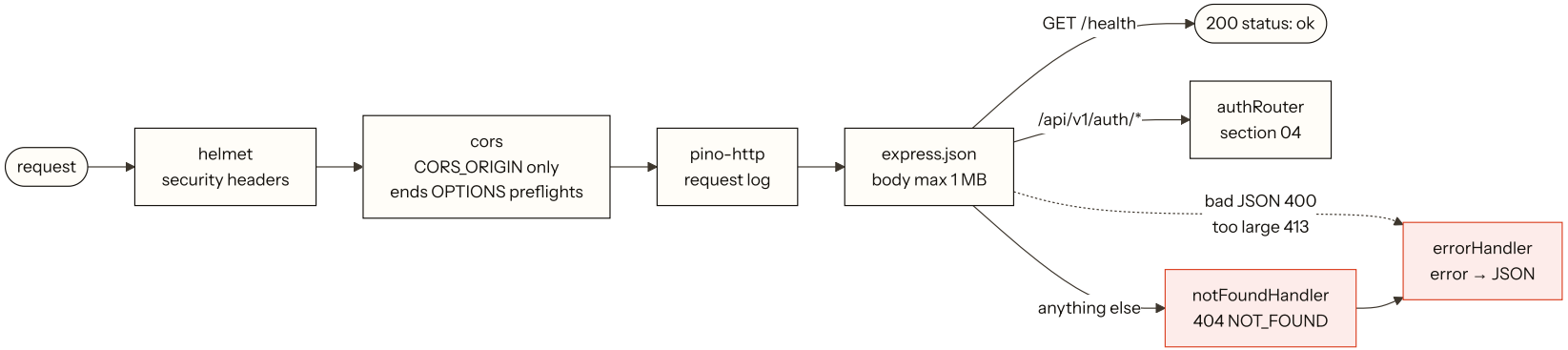


RULES

- Dependencies point one way: routes → controller → service → model.
- Controllers own HTTP. Services never see `req` or `res`.
- Services fail by throwing `AppError`. No try/catch in controllers: Express 5 forwards rejected promises.

03 GLOBAL MIDDLEWARE

What every request passes through before any route code runs (`app.ts`).



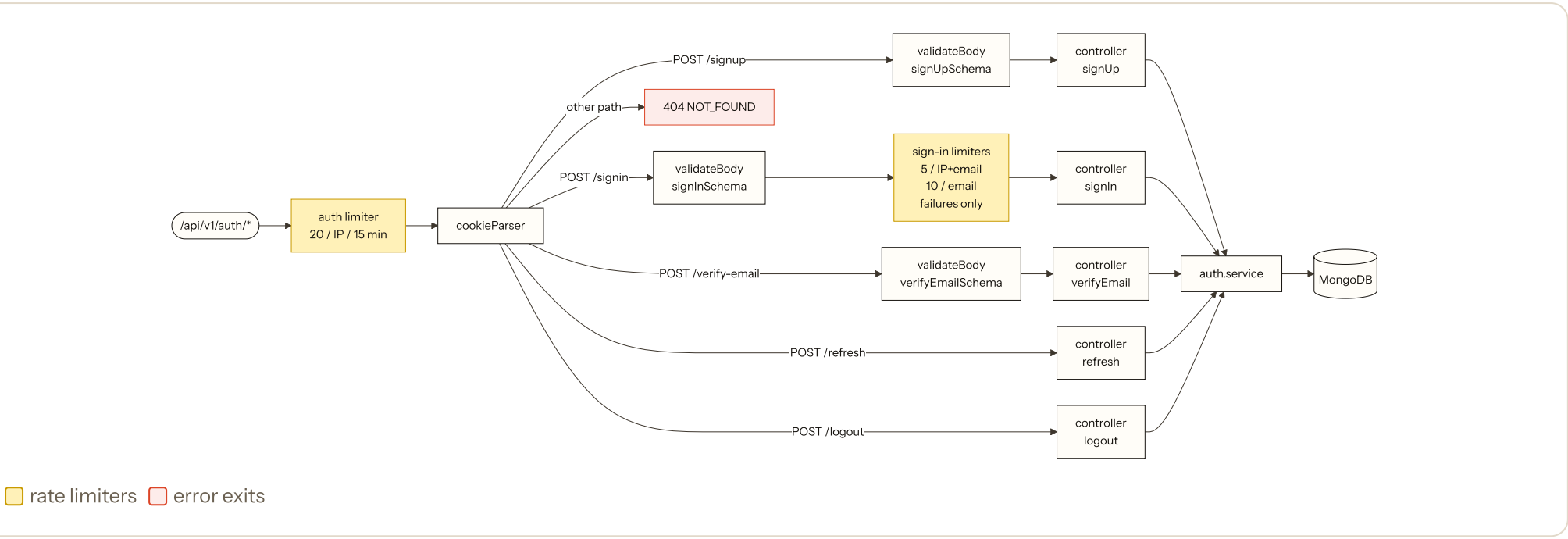
error exits. Middleware runs in registration order; any step can stop the request with `next(error)` .

WHAT EACH STEP IS FOR

MIDDLEWARE	WHAT IT DOES	WHY IT MATTERS
helmet	Adds security headers: CSP, HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy…	Browser-side hardening on every response, for free.
cors	Allows only `CORS_ORIGIN` , with credentials. Answers `OPTIONS` preflights itself.	Other sites can't call the API from a user's browser. Preflights never reach a rate limiter.
pino-http	Logs method, URL, status and response time; gives handlers `req.log` .	Every request is traceable, without logging bodies or secrets.
express.json	Parses JSON bodies up to 1 MB.	Malformed (400) and oversized (413) bodies are rejected before any logic runs.
notFound + errorHandler	Turns unknown routes and every thrown error into JSON.	The client always gets one consistent error shape (section 11).

04 AUTH ROUTER

How every `/api/v1/auth` route is guarded (`auth.routes.ts`).



KEY POINTS

- The auth limiter counts **every** request under `/auth`, including invalid bodies and unknown paths.
- `validateBody` parses with zod and replaces `req.body` with the clean value: trimmed, email lowercased, unknown keys dropped.
- The sign-in limiters run **after** validation, so `A@x.com` and `a@x.com` share one counter.
- `cookieParser` is mounted on this router only: the `refresh_token` cookie is scoped to `/api/v1/auth`, so no other route receives it.
- Any step can fail into `errorHandler`: 429 from a limiter, 422 from validation, 400 or 401 from the service.

05 ROUTES AND RESPONSES

Every endpoint, what runs in front of it, and what it can answer.

ROUTE	ACCESS	MIDDLEWARE CHAIN (AFTER GLOBAL)	SUCCESS	ERRORS
`GET /health`	public	none	200 `{status:"ok"}`	none
`POST /api/v1/auth/signup`	public	auth limiter → cookieParser → validateBody(signUpSchema) → signUp	202 `{message}`, same for a new or taken email	422 · 429
`POST /api/v1/auth/verify-email`	emailed link	auth limiter → cookieParser → validateBody(verifyEmailSchema) → verifyEmail	200 `{message}`, sets `emailVerifiedAt`	400 INVALID_VERIFICATION_TOKEN · 422 · 429
`POST /api/v1/auth/signin`	public	auth limiter → cookieParser → validateBody(signInSchema) → 2 sign-in limiters → signIn	200 `{user, accessToken}` + `refresh_token` cookie	401 INVALID_CREDENTIALS · 422 · 429
`POST /api/v1/auth/refresh`	`refresh_token` cookie	auth limiter → cookieParser → refresh	200 `{user, accessToken}` + new `refresh_token` cookie	401 INVALID_REFRESH_TOKEN · 429
`POST /api/v1/auth/logout`	public	auth limiter → cookieParser → logout	204, cookie cleared, always	429

RESPONSE SHAPES

200 · SIGN-IN AND REFRESH

```
{
  "user": { "id": "67…", "name": "Ada",
    "email": "a@x.io", "role": "user" },
  "accessToken": "eyJhbGci…"
}
+ Set-Cookie: refresh_token=…; HttpOnly;
  Secure; SameSite=Strict;
  Path=/api/v1/auth
```

4XX / 5XX · ANY ERROR

```
{
  "code": "INVALID_CREDENTIALS",
  "message": "Invalid email or password"
}
```

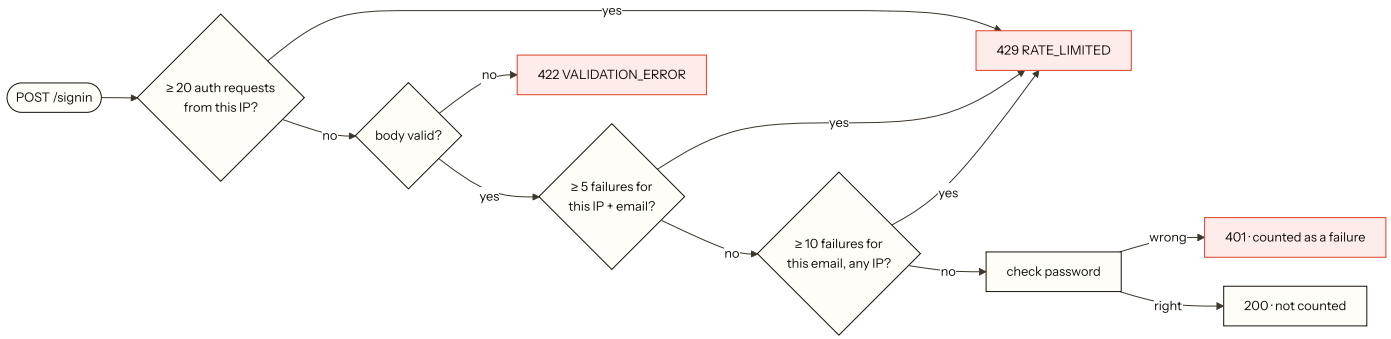
422 · VALIDATION

```
{
  "code": "VALIDATION_ERROR",
  "errors": {
    "formErrors": [],
    "fieldErrors": { "password": [
      "Too small: expected string…" ] }
  }
}
```

06 RATE LIMITS

Three layers, each stopping a different kind of password guessing.

LIMITER	APPLIES TO	COUNTED PER	LIMIT	WHAT COUNTS	STOPS
Auth limiter	every request under `/api/v1/auth`	client IP	20 / 15 min	every request	one machine spraying requests
Sign-in · IP + email	`POST /signin`, after validation	IP + normalised email	5 / 15 min	failures only (status ≥ 400)	guessing one password from one machine
Sign-in · email	`POST /signin`, after validation	normalised email, any IP	10 / 15 min	failures only	guessing one password from many machines

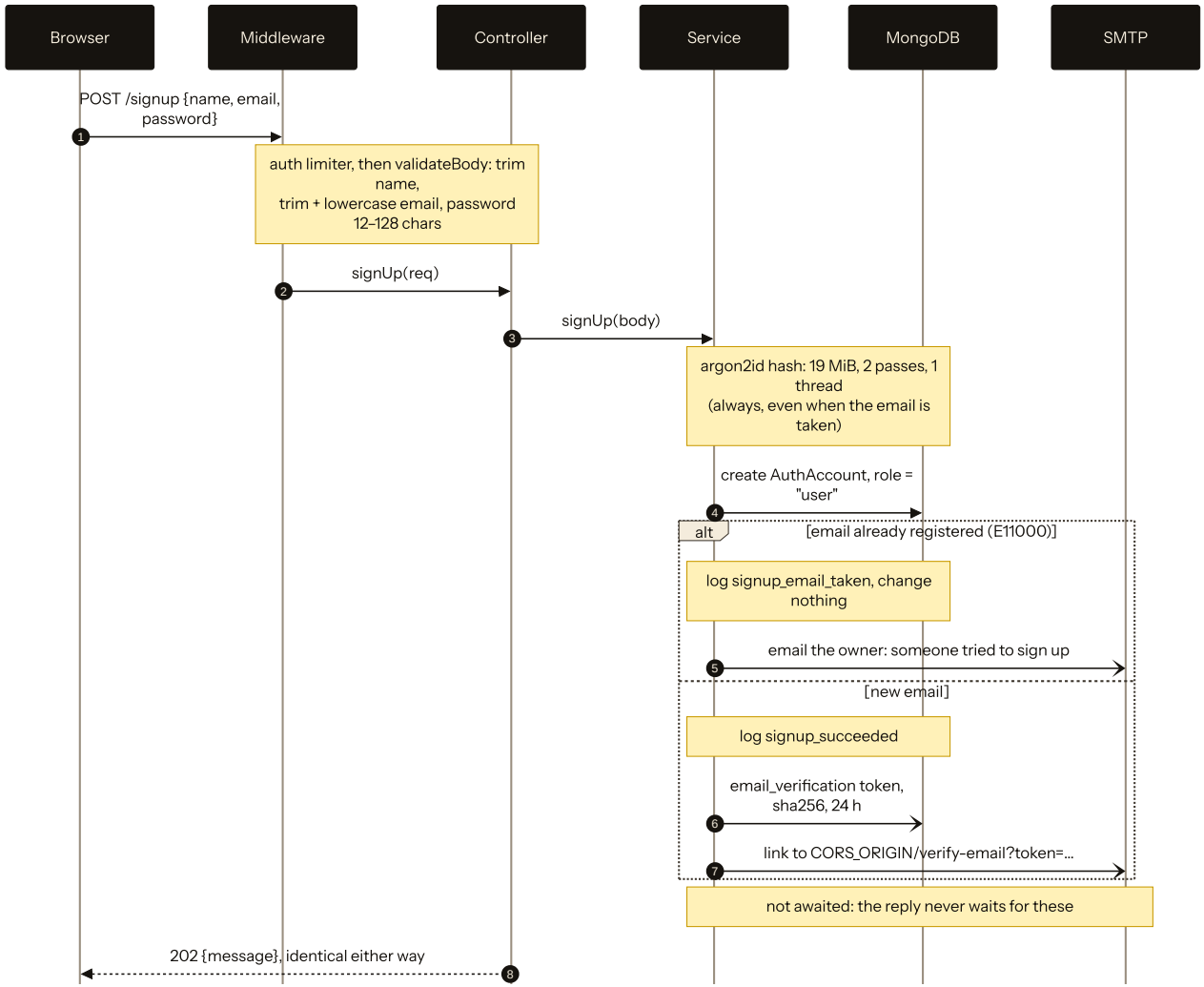


HOW THEY'RE IMPLEMENTED

`express-rate-limit` with fixed 15-minute windows; responses carry the IETF `RateLimit` and `RateLimit-Policy` headers. Counters live **in process memory**: they reset on restart and are not shared between instances. TRADE-OFF The email-only limiter lets anyone lock an email out of sign-in for up to 15 minutes.

07 SIGN-UP FLOW

`POST /api/v1/auth/signup`, step by step.

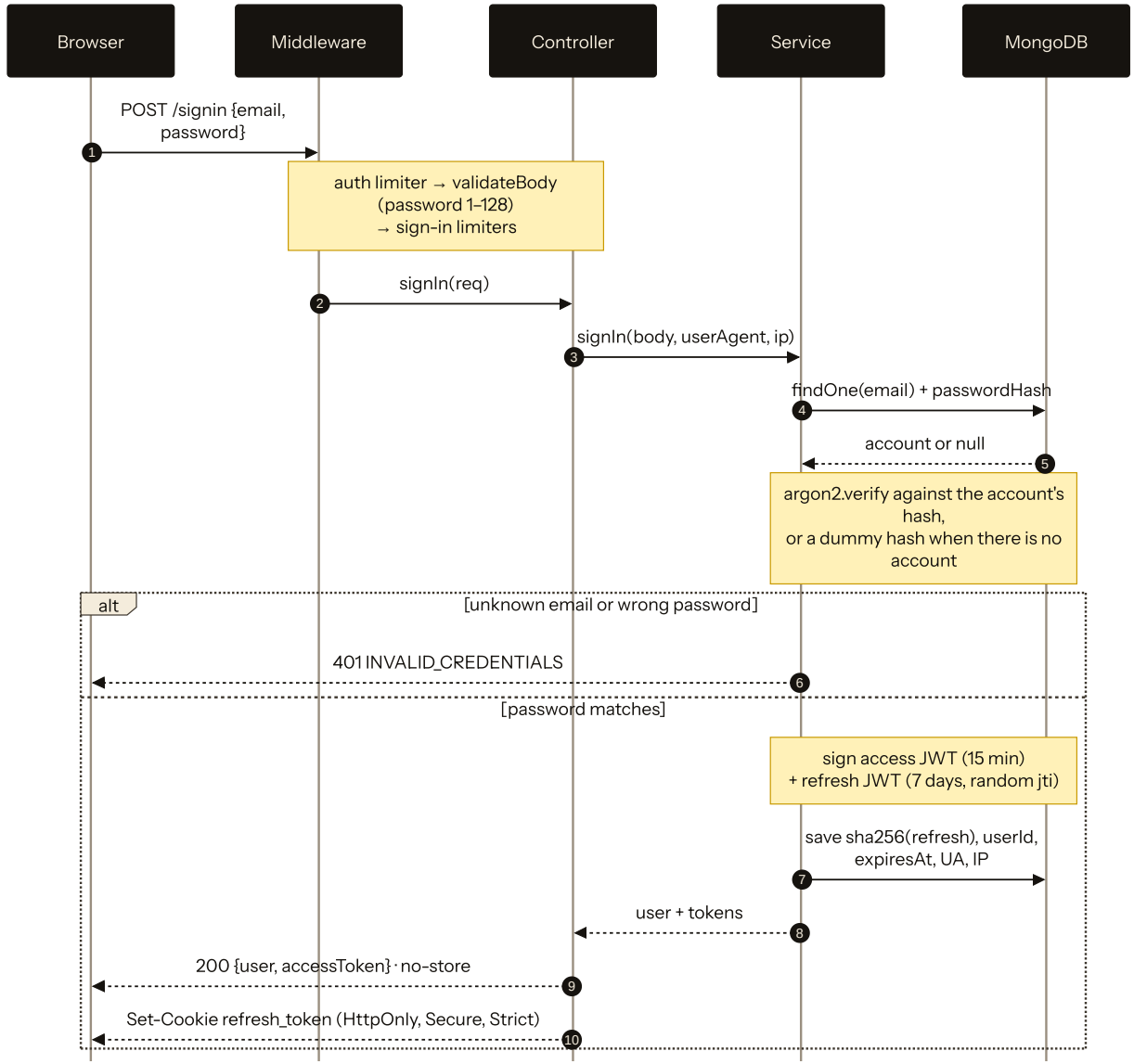


KEY POINTS

- The email is trimmed and lowercased first, so `Ada@X.com` and `ada@x.com` are one account.
- zod drops unknown keys: sending `"role": "admin"` does nothing.
- Passwords are hashed with argon2id at the OWASP minimum cost.
- No `exists` pre-check: the unique email index decides, which also covers two sign-ups racing.
- A taken email gets the same 202 after the same work (one hash, one insert). Token and emails are not awaited, so timing reveals nothing.
- A taken email warns the owner but never gets a verification link (it would verify whoever registered first). No email includes the name.

08 SIGN-IN FLOW

`POST /api/v1/auth/signin`, step by step.

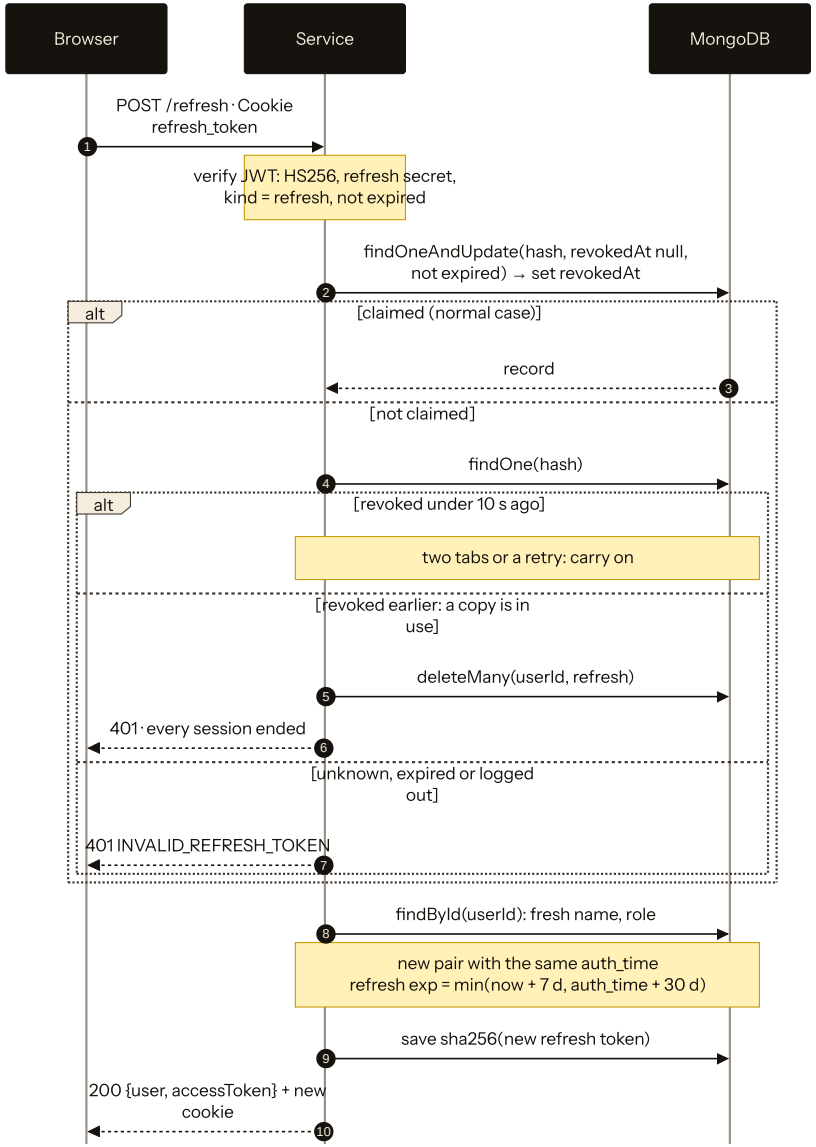


KEY POINTS

- Unknown email and wrong password get the **same** 401 body, and the dummy hash makes them take the same time.
- The access token goes in the body. The refresh token goes **only** in the cookie.
- Only a sha256 of the refresh token is stored, with user-agent and IP for a future "your devices" list.
- JWT `exp`, database `expiresAt` and cookie `Expires` come from one Date.
- Logged as `signin_failed {reason}` or `signin_succeeded {userId}`, never with the email.

09 REFRESH AND LOGOUT

`POST /api/v1/auth/refresh` and `/logout`: both read only the `refresh_token` cookie.



REFRESH

- **Rotation:** every refresh revokes the old token and issues a new one. The atomic update means two requests with one token can't both claim it.
- **Reuse detection:** only rotation sets `revokedAt`, so a revoked token coming back means someone kept a copy. Logged as `refresh_token_reused`.
- **10-second grace:** covers two tabs loading at once, like Okta's grace period and Auth0's reuse interval.
- **Session cap:** ends after 7 idle days, or 30 days after sign-in at most.

LOGOUT

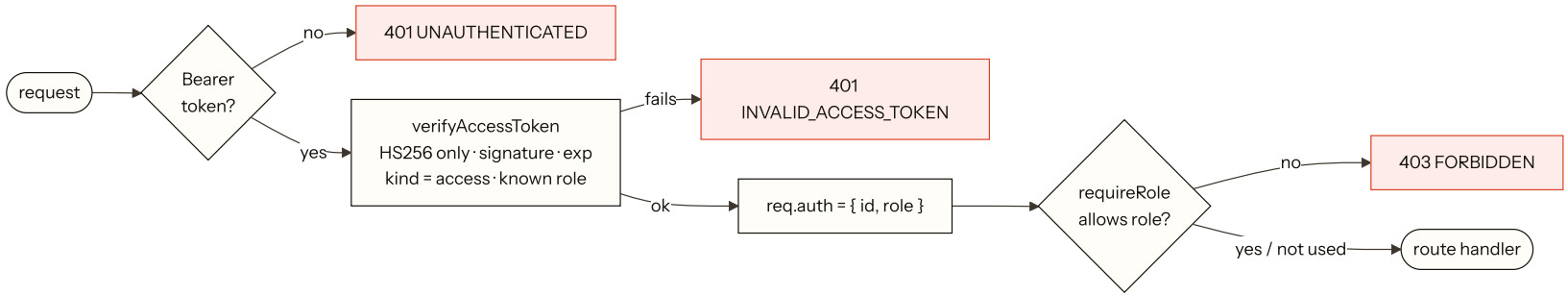
- Deletes this device's record (not revoke, so it never looks like reuse), clears the cookie with the same options, always 204.
- No access token needed: it may already have expired.

10 TOKENS AND PROTECTED ROUTES

Two tokens with different jobs, and the check that guards private routes.

	ACCESS TOKEN	REFRESH TOKEN
Format	JWT, HS256, signed with `JWT_ACCESS_SECRET`	JWT, HS256, signed with `JWT_REFRESH_SECRET`
Claims	`sub` (user id), `role`, `kind: "access"`, `auth_time` (sign-in time), `jti`, `iat`, `exp`	same, with `kind: "refresh"`
Lifetime	15 minutes (`JWT_ACCESS_TOKEN_TTL`)	7 days from the last refresh (`JWT_REFRESH_TOKEN_TTL_DAYS`), 30 days after sign-in at most
Client keeps it	in memory (React state), never localStorage	cookie `refresh_token` : HttpOnly, Secure, SameSite=Strict, Path=/api/v1/auth
Sent as	`Authorization: Bearer <token>`	automatically by the browser, only to `/api/v1/auth/*`
Server stores	nothing (stateless)	sha256 hash in `authtokens` ; a TTL index deletes it after expiry
Revocable	no; it stays valid until it expires, even after logout	yes: revoked on rotation, deleted on logout or reuse

PROTECTED ROUTES BUILT, NOT MOUNTED YET

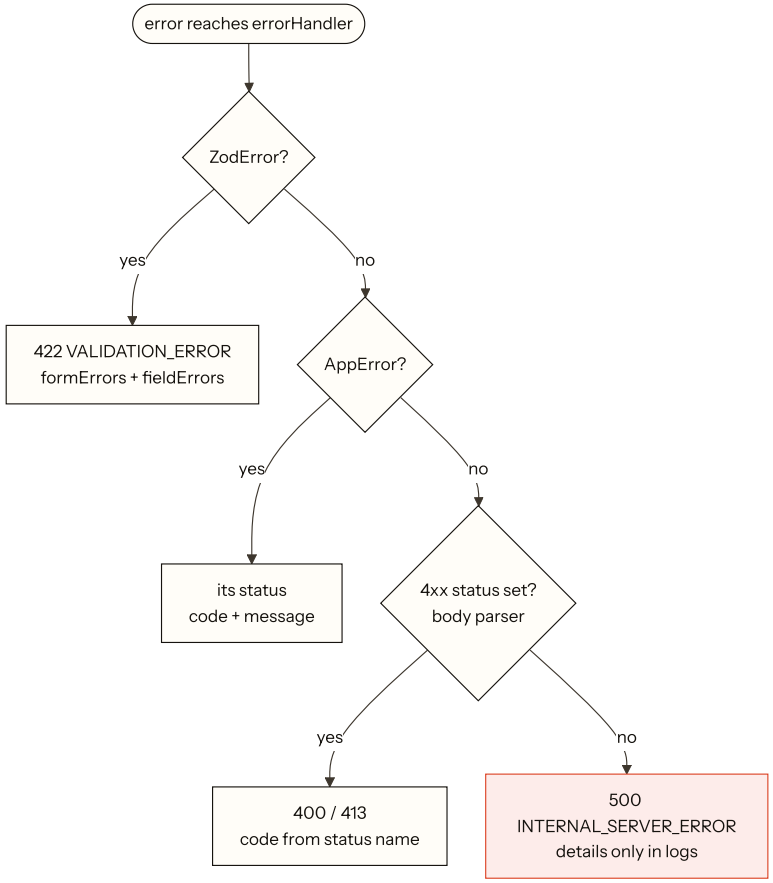


Usage: `router.get("/admin/x", requireAuth, requireRole("admin"), handler)`. Pinning HS256 blocks algorithm-confusion attacks; the `kind` claim stops a refresh token being used as an access token.

11 ERROR HANDLING

One place turns every failure into the same JSON shape.

Services fail by throwing `new AppError(status, message, CODE)`. Every error, from any layer, ends in `errorHandler`. 4xx errors are logged at `warn`, 500s at `error` with the stack.



CODE	STATUS	RAISED BY
VALIDATION_ERROR	422	`validateBody` (zod)
INVALID_CREDENTIALS	401	`signIn` : unknown email or wrong password
RATE_LIMITED	429	any limiter in `auth.routes.ts`
UNAUTHENTICATED	401	`requireAuth` : no Bearer header
INVALID_ACCESS_TOKEN	401	`verifyAccessToken`
FORBIDDEN	403	`requireRole`
NOT_FOUND	404	`notFoundHandler`
INVALID_REFRESH_TOKEN	401	`refresh` : cookie missing, invalid, expired, logged out or reused
INVALID_VERIFICATION_TOKEN	400	`verifyEmail` : link unknown, expired or already used
INTERNAL_SERVER_ERROR	500	anything unexpected

12 DATA MODEL

Two collections: accounts, and the tokens issued to them.

AuthAccount			
ObjectId	_id	PK	
string	name		trimmed, max 100
string	email	UK	trimmed, lowercase
string	passwordHash		argon2id, select false
string	role		user or admin
date	emailVerifiedAt		null until verified
date	createdAt		
date	updatedAt		

owns

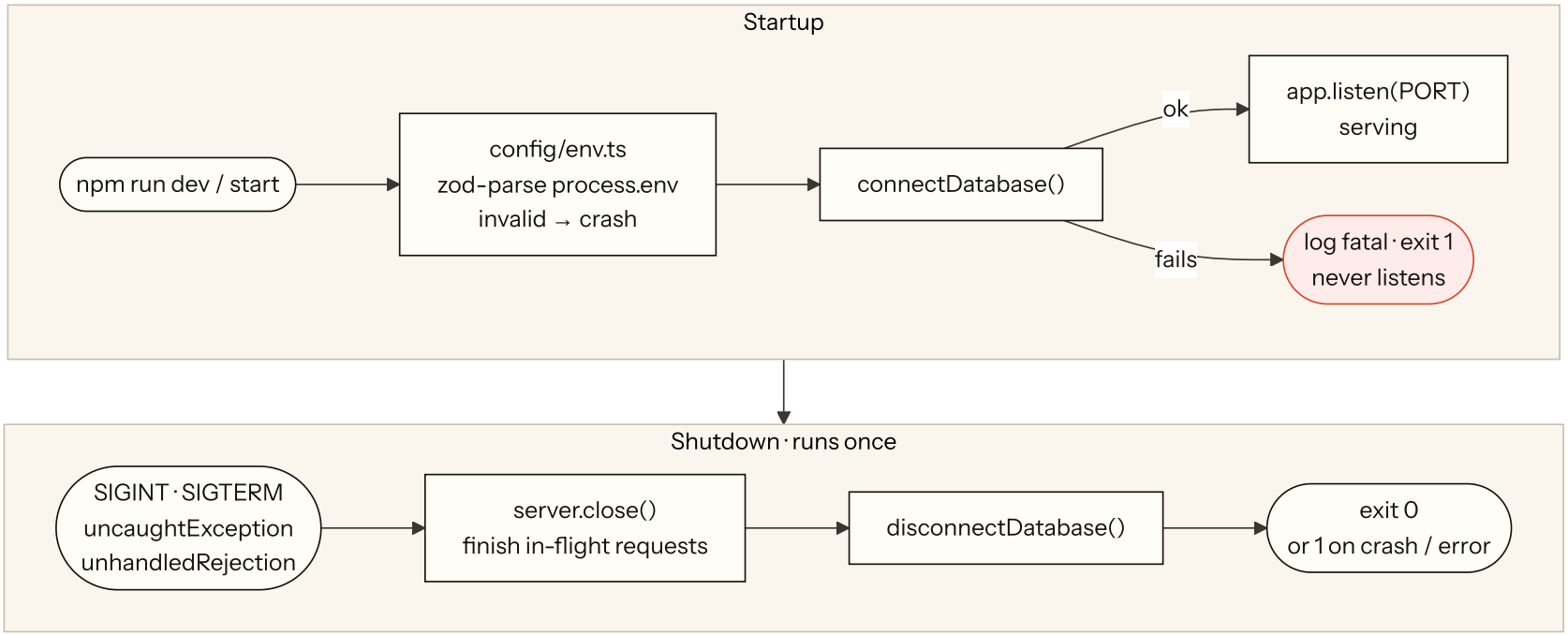
AuthToken			
ObjectId	_id	PK	
string	tokenHash	UK	sha256, select false
ObjectId	userId	FK	AuthAccount
string	type		refresh, password_reset, email_verification
date	expiresAt		TTL index
date	revokedAt		set only by rotation
string	userAgent		max 512
string	ipAddress		max 45

INDEXES AND WHY

- `authaccounts.email` unique: one account per email; catches sign-up races.
- `authtokens.tokenHash` unique: finds a presented token in one lookup.
- `authtokens {userId, type}` : reuse detection deletes every refresh token of one user; also serves `userId` -only queries.
- `authtokens.expiresAt` TTL: MongoDB deletes expired tokens itself (about once a minute).
- `select: false` hides `passwordHash` and `tokenHash` unless a query asks for them.

13 STARTUP AND SHUTDOWN

How the process comes up, and how it goes down cleanly (`server.ts`).



KEY POINTS

- MongoDB connects **before** the port opens, so the API never takes traffic it can't serve.
- A bad or missing env variable stops the process at import time with a zod error.
- On SIGINT/SIGTERM the server stops accepting, lets in-flight requests finish, then closes MongoDB.
- Crashes take the same path and exit with code 1.

14 SECURITY CONTROLS

Each threat, what stops it, and where that lives in the code.

THREAT	CONTROL	WHERE
Stolen database → passwords	argon2id, 19 MiB memory, 2 passes, 1 thread (OWASP minimum). Hash not selected by default.	`auth.service.ts`, `auth.model.ts`
Stolen database → sessions	Only sha256 hashes of refresh tokens are stored.	`auth.tokens.ts` · hashToken
Password guessing	Three layered rate limits (section 06).	`auth.routes.ts`
Email enumeration	Sign-in: identical 401, plus a dummy argon2 verify so timing matches. Sign-up: identical 202, always one hash and one insert, no pre-check.	`auth.service.ts` · signIn, signUp
Stolen refresh token	Rotated on every use. A rotated token coming back (after a 10-second grace) deletes every refresh token of that user. Sessions end 30 days after sign-in regardless.	`auth.service.ts` · refresh
Tokens leaking: XSS, caches	Refresh token is HttpOnly. Access token lives only in memory and expires in 15 minutes. `Cache-Control: no-store` on token responses.	`auth.controller.ts`, front-end `session.tsx`
CSRF and login CSRF	SameSite=Strict cookie scoped to `/api/v1/auth`. JSON-only bodies force a CORS preflight that only `CORS_ORIGIN` passes.	`app.ts`, `auth.controller.ts`
Forged or confused JWTs	HS256 pinned on verify, a separate secret per token kind, `kind` and role checked.	`auth.tokens.ts`
Phishing via our emails	Plain text, and the user's name is never included.	`auth.service.ts` · signUp
Privilege escalation, NoSQL injection	zod drops unknown keys, so `role` can't be set at sign-up, and requires strings, so objects like `{"$gt": ""}` get 422.	`auth.validation.ts`
Oversized input, CPU abuse	1 MB body limit, max lengths on every field, password capped at 128 characters before hashing.	`app.ts`, `auth.validation.ts`
Secrets in logs	pino redacts password and token fields; request logs keep only method and URL; auth events log userId, never email.	`config/logger.ts`
Clickjacking, MIME sniffing	helmet default headers.	`app.ts`
Misconfiguration	Env validated at startup; bad values crash immediately.	`config/env.ts`

15 CONFIGURATION AND TESTING

Every setting the server reads, and how its behaviour is checked.

Loaded with Node's built-in `--env-file=.env` and validated by zod in `config/env.ts`. Code reads `config.X`, never `process.env`. A new variable goes in the schema and in `.env.example`.

VARIABLE	RULE	DEFAULT	USED FOR
NODE_ENV	development · test · production	development	log level; `test` silences logs
PORT	1–65535	3000	`app.listen` (dev: 8080)
MONGODB_URI	required		cluster + database `ticker-rush-dev`
CORS_ORIGIN	URL, required		the one allowed browser origin
JWT_ACCESS_SECRET	≥ 15 chars GAP		signs access tokens
JWT_REFRESH_SECRET	≥ 15 chars GAP		signs refresh tokens
JWT_ACCESS_TOKEN_TTL	time span	15m	access token lifetime
JWT_REFRESH_TOKEN_TTL_DAYS	integer 1–30	7	refresh lifetime, cookie, DB TTL
SMTP_HOST, _USER, _PASS	required		mail server login (dev: Ethereal)
SMTP_PORT	1–65535	587	465 = TLS at once, else STARTTLS
MAIL_FROM	text	no-reply@tickerrush.dev	sender of every email

TESTING · NPM TEST

- Node's built-in `node:test` over `src/**/*.test.ts`, with `NODE_ENV=test`.
- Starts the real app on a random port and calls it with `fetch`; no supertest.
- Same `MONGODB_URI`, but `dbName` is overridden to `ticker-rush-test`. Tests delete what they create.
- No email is sent: with `NODE_ENV=test` the mailer uses nodemailer's `jsonTransport`.

COVERED TODAY

- Sign-in: token, cookie flags, hash-only storage, identical 401s, 429 on the 6th failure.
- Sign-up: a taken email looks identical to a new one; a new account gets one 24-hour verification token, stored as a hash.
- Refresh: rotation, grace window, reuse ends every session, 30-day cap.
- Logout: 204 with or without a cookie; the token stops working.

16 KNOWN GAPS AND NEXT STEPS

What isn't safe or finished yet, and the plan for each.

GAP	RISK TODAY	PLAN
JWT secrets only need 15 chars	A short HS256 secret can be brute-forced offline from any token.	Raise the zod minimum to 32, as `.env.example` already says.
Rate limits in memory	Fine for one process; each extra instance multiplies the real limit.	Move to a Redis store when running more than one instance.
`trust proxy` **not set**	Behind Nginx or a load balancer every client shares the proxy's IP, so one user can rate-limit everyone.	Set it to the proxy hop count at deploy time.
Email verification unfinished	Links can be confirmed, but sign-in doesn't check `emailVerifiedAt` yet. No password reset either.	Block unverified sign-in and add a resend endpoint, then password reset.
Sign-up can email any address	Only the auth limiter (20 per IP per 15 min) slows inbox flooding.	Per-email limit on sign-up and resend.
Emails are fire-and-forget TRADE-OFF	Lost if the mail server fails or the process stops first; only logged.	BullMQ queue on Redis, with retries.
Access token after logout TRADE-OFF	Stays valid for up to 15 minutes after logout or reuse detection.	Keep it short-lived. Add a `jti` denylist only if a route needs instant revocation.
Lost refresh response TRADE-OFF	If the new cookie never arrives and the browser retries after the 10-second grace, reuse detection signs the user out everywhere.	Accepted: safer than letting an old token work indefinitely.
Refresh shares the auth limiter TRADE-OFF	Every full page load spends 1 of the 20 requests per 15 minutes.	Give `/refresh` its own, higher limit if users hit 429.
Per-email lockout TRADE-OFF	An attacker can block a user's sign-in for up to 15 minutes.	Accepted for now; revisit with CAPTCHA or progressive delays.

Updating this document: edit `server/docs/architecture.html`, run `npm run docs:pdf` from `server/`, and commit both files. Update it whenever routes, middleware, security, rate limits, tokens, models, error codes or env variables change.